

CS-412 Fuzzing Lab Report

Octave Charrin, Emanuele Martelli, David Pulido, Gabriele Stentella

January 17, 2026

Introduction

Students:

- Octave Charrin, 404884
- Emanuele Martelli, 404916
- David Pulido, 390127
- Gabriele Stentella, 404913

For the fuzzing lab we chose the libpng project <https://github.com/pnggroup/libpng>. If not stated otherwise, all the commands written in this document are executed from the root of the oss-fuzz <https://github.com/google/oss-fuzz> directory.

Our submission repository can be found at: <https://github.com/gabrielestentella/CS412-Fuzzing-Lab-libpng/>.

1 Part 1

Libpng only has one harness called `libpng_read_fuzzer`.

1.1 Run with corpus

First, we clone both the oss-fuzz directory and the libpng directory. Note here that we need to use the correct version of libpng since a bug has been introduced in one of the last commits on the main project repository (see <https://github.com/pnggroup/libpng/issues/678> for more details). We then modify the Dockerfile in oss-fuzz so that when we build the fuzzer image, docker uses the previously cloned version of libpng, instead of freshly cloning the last version of libpng directly from the github repository. We use the following diff file, located in `part1/oss-fuzz.diff` (using `git apply ../oss-fuzz.diff`) to modify the Dockerfile:

```
diff --git a/projects/libpng/Dockerfile b/projects/libpng/Dockerfile
index 6f281cd55..36ffea5c3 100644
--- a/projects/libpng/Dockerfile
+++ b/projects/libpng/Dockerfile
@@ -19,6 +19,7 @@ RUN apt-get update && \
     apt-get install -y make autoconf automake libtool zlib1g-dev

 RUN git clone --depth 1 https://github.com/madler/zlib.git
-RUN git clone --depth 1 https://github.com/pnggroup/libpng.git
```

```

+# RUN git clone --depth 1 https://github.com/pnggroup/libpng.git
+ADD libpng libpng
  RUN cp libpng/contrib/oss-fuzz/build.sh $SRC
  WORKDIR libpng

```

We then move the correct version of libpng into the oss-fuzz directory (with `mv ../libpng projects/libpng/`) to make it accessible in the container. Ultimately, we use the following commands to build and run the fuzzer using the provided seed corpus:

```

python3 infra/helper.py build_image --no-pull libpng
python3 infra/helper.py build_fuzzers --clean libpng \
  projects/libpng/libpng/
mkdir libpng_read_fuzzer_with_corpus
python3 infra/helper.py run_fuzzer --corpus-dir \
  ./libpng_read_fuzzer_with_corpus libpng \
  libpng_read_fuzzer -- -max_total_time=14400

```

After four hours of fuzzing, the process terminates and we generate the coverage report with the following commands:

```

python3 infra/helper.py build_fuzzers --sanitizer coverage \
  --clean libpng projects/libpng/libpng
python3 infra/helper.py coverage --no-serve --fuzz-target \
  libpng_read_fuzzer --corpus-dir \
  ./libpng_read_fuzzer_with_corpus libpng

```

The html file of the coverage report is generated at the path `oss-fuzz/build/out/libpng/report/linux/report.html`. We moved the coverage report of the fuzzing campaign with seed corpus at the path `part1/report/w_corpus/linux/report.html`.

1.2 Run without corpus

To run without seed corpus, we need to modify the build script so that it does not create the seed corpus directory. Moreover, we need to modify the Dockerfile so that when we build the fuzzer image, docker uses the libpng directory with the modified script, and not the cloned one. We apply the following diff file, located in `part1/project.diff`, using `git apply ../project.diff` (from the libpng directory) to modify the build script:

```

diff --git a/contrib/oss-fuzz/build.sh b/contrib/oss-fuzz/build.sh
index 7b8f026..c7d0fce 100755
--- a/contrib/oss-fuzz/build.sh
+++ b/contrib/oss-fuzz/build.sh
@@ -43,8 +43,8 @@ $CXX $CXXFLAGS -std=c++11 -I. \
    -lFuzzingEngine .libs/libpng16.a -lz

# add seed corpus.
-find $SRC/libpng -name "*.png" | grep -v crashers | \
-  xargs zip $OUT/libpng_read_fuzzer_seed_corpus.zip
+# find $SRC/libpng -name "*.png" | grep -v crashers | \
+#    xargs zip $OUT/libpng_read_fuzzer_seed_corpus.zip

cp $SRC/libpng/contrib/oss-fuzz/*.dict \
  $SRC/libpng/contrib/oss-fuzz/*.options $OUT/

```

Again, we move this modified version of libpng to the folder dedicated to the libpng project in the oss-fuzz directory, and modify the Dockerfile so that it uses this modified version of libpng (`git apply ../oss-fuzz.diff`).

After these modifications, we can run the fuzzer using the following commands:

```
python3 infra/helper.py build_image --no-pull libpng
python3 infra/helper.py build_fuzzers --clean libpng \
    projects/libpng/libpng/
mkdir libpng_read_fuzzer_without_corpus
python3 infra/helper.py run_fuzzer --corpus-dir \
    ./libpng_read_fuzzer_without_corpus libpng \
    libpng_read_fuzzer -- -max_total_time=14400
```

After four hours of fuzzing, the process terminates and we generate the coverage report with the following commands:

```
python3 infra/helper.py build_fuzzers --sanitizer coverage \
    --clean libpng projects/libpng/libpng
python3 infra/helper.py coverage --no-serve --fuzz-target \
    libpng_read_fuzzer --corpus-dir \
    ./libpng_read_fuzzer_without_corpus libpng
```

The html file of the coverage report is generated at the path `oss-fuzz/build/out/libpng/report/linux/report.html`. We moved the coverage report of the fuzzing campaign without seed corpus at the path `part1/report/w_o_corpus/linux/report.html`.

1.3 Differences in coverage between the two runs

The total line coverage after running the fuzzer for four hours using the provided seed corpus is 40.80% (48.50% function coverage), while it is 23.61% (32.75% function coverage) for the run without seed corpus.

Files like `png.c`, `pngerror.c`, `pngread.c`, `pngtran.c`, `pngutil.c`, `pngset.c` show substantial increases in all coverage metrics (line, function, and region) when using the default corpus compared to the empty corpus. Coverage remains very low across all metrics in both scenarios for `pngget.c`; for `pngtrans.c` the improvement is significant, but the final coverage remains low.

For `pngmem.c` and `pngrio.c` the coverage is high also for the run with empty seed corpus. Indeed, the first file contains the functions dedicated to memory management: Even with an empty corpus, the act of creating and potentially destroying the `png_struct` instances triggers the default memory allocation and deallocation paths within `pngmem.c`. The file `pngrio.c` contains only two functions to read data, and setting up the input mechanism is necessary before any PNG data processing can occur, regardless of the seed corpus.

One of the files with the biggest improvement is `pngread.c`, this file contains the core functions responsible for parsing and processing PNG image data. Hence, with a seed corpus of PNG files with different characteristics like different bit depths, filter methods, compression levels and chunk combination it is possible to significantly increase the coverage.

2 Part 2

The current implementation of the fuzzer is designed to evaluate the behavior of libpng’s PNG reading functionality. It systematically analyzes the library’s response to a variety of input files, including both well-formed and malformed PNG images. This fuzzer operates by accepting arbitrary sequences of bytes as input, verifying the presence of a valid PNG signature, and then invoking libpng’s primary image reading interfaces. In doing so, it applies common image transformations such as grayscale-to-RGB conversion and image data expansion: techniques commonly employed by modern web browsers. It evaluates both the standard reading path and the simplified read API provided by libpng.

The existing fuzzer provides coverage for several key functionalities within the PNG reading pipeline. These include validation of PNG file headers and signature fields, parsing and handling of essential PNG chunks, application of typical image transformations, and fundamental error handling mechanisms along the read paths. However, the fuzzer does not thoroughly exercise several critical reading functionalities, indeed the line coverage of the `pngread.c` module is only 28.36%.

Below, we identify and analyze two significant uncovered code regions, justify their importance, and explain why the existing harness fails to cover them.

2.1 Uncovered Region 1: Colormap Processing and Metadata Accessor Functions

This region encompasses libpng’s colormap-related functions and metadata accessor functions, which are essential for processing color data and retrieving image metadata, making them critical targets for fuzz testing.

Functionality: The colormap functions handle the interpretation, transformation, and application of color data in PNG images. For instance, `png_create_colormap_entry` performs low-level color conversions, such as gamma correction and encoding-specific transformations, to create colormap entries. Similarly, `set_file_encoding` determines the image’s encoding format based on gamma metadata, guiding subsequent color data processing. Higher-level operations are managed by `png_image_read_colormap`, which builds a complete colormap for various color types and transparency settings, relying on auxiliary routines. The `png_image_read_and_map` function transforms pixel data into colormap indices, supporting interlaced formats and alpha composition, while `png_image_read_colormapped` orchestrates the workflow, integrating chunk handling, format validation, and pixel processing.

Complementing these, `png_get_*` accessor functions (e.g., `png_get_image_width`, `png_get_image_height`) extract metadata from a parsed `png_struct`. These functions retrieve diverse metadata, including image geometry, palette tables, color profiles, textual chunks, and resource limits. Executed post-parsing, they are accessible in applications like browsers, thumbnail generators, and document viewers. They perform sensitive operations, such as forwarding internal pointers with attacker-supplied lengths (e.g., `png_get_rows` returns `row_pointers` with image height), managing doubly indirectioned string tables (e.g., `png_get_text` returns a `png_charpp` list), and converting integers to floats (e.g., `png_get_pixel_aspect_ratio` divides `y_pixels_per_unit` by `x_pixels_per_unit`). These operations risk memory safety issues, such as buffer overflows or uninitialized memory accesses.

Justification of Significance: The colormap functions involve intricate, data-dependent logic with numerous conditional branches and bit-level operations, creating a broad at-

tack surface. They allocate and manipulate memory buffers, increasing the risk of overflows, and perform mathematical operations prone to errors like underflows or division-by-zero. Handling edge cases, such as varying bit depths and transparency, further amplifies their complexity, making them susceptible to vulnerabilities like buffer overflows, memory leaks, null pointer dereferences, and logical errors in color transformations.

The accessor functions are equally critical, as they operate on parsed `png_struct` data, exposing metadata to end-user applications. A memory safety error in these functions could be triggered by a malicious image in a browser or viewer, potentially enabling exploits. For example, `png_get_rows` returns attacker-controlled lengths, risking buffer overflows, while `png_get_text` handles complex string tables vulnerable to mismanagement. Given libpng’s widespread use in image processing, vulnerabilities in this region could have severe security implications, which emphasizes the need for thorough fuzzing to close this “post-parse” attack surface.

Shortcoming of Existing Harnesses: The current harness prioritizes basic image parsing and simple transformations, overlooking colormap processing and metadata access. The seed corpus lacks PNGs with complex color types, transparency, or ancillary chunks (e.g., gamma or text metadata) required to trigger these functions. Consequently, only `png_get_IHDR` and `png_get_rowbytes` are covered, accounting for less than 3% of accessor functions. The harness does not explicitly invoke colormap-related APIs or accessors beyond header validation, restricting coverage of these code paths and leaving significant portions of `pngread.c` unexplored.

2.2 Uncovered Region 2: Write Pipeline

The second overlooked surface resides in the main writer pipeline corresponding to the set of entry points that turn an application-supplied pixel buffer plus associated metadata into a valid PNG bit-stream.

Functionality: The write pipeline includes functions like `png_create_write_struct`, `png_write_row`, and `png_write_image`. These routines perform complex operations, such as non-trivial pointer arithmetic (e.g., `row_end = output_row + image->width * (channels + 1)` in `png_write_image`) and bulk memory copies (e.g., `memcpy(png->row_buf + 1, row, row_info.rowbytes)` in `png_write_row`). This pipeline is invoked when applications, such as email clients generating thumbnails or chat apps downscaling images, re-encode untrusted user data.

Justification of Significance: The write pipeline is significant because it processes potentially attacker-controlled data in common use cases, such as image re-encoding. Memory safety bugs, such as buffer overflows or invalid pointer dereferences, could be exploited in scenarios where untrusted images are processed, leading to severe vulnerabilities. The pipeline’s heavy reliance on pointer arithmetic and memory operations increases the likelihood of implementation errors, which could propagate to end-user applications. As libpng is integral to many image-processing workflows, uncovering flaws in this region is crucial for enhancing security.

Shortcoming of Existing Harnesses: Naturally, none of this code is reached by the base `libpng_read_fuzzer`. Indeed, as explained before, that harness focuses merely on read and transform operations: The current build script for the fuzzing campaign on oss-fuzz, compiles the library without the flag `PNG_WRITE_SUPPORTED`. Consequently, the introspector report shows virtually 0% line and function coverage for `pngwrite.c` (and related files such as `pngwio.c` and `pngwutil.c`), whereas files involved in reading reach beyond 40% after four hours of fuzzing. Thus, by introducing a dedicated write

fuzzer we can unlock an entire half of the library that was previously invisible to the original harness.

3 Part 3

3.1 Uncovered Region 1: Colormap Processing and Metadata Accessor Functions

Here we present the improvements made to the `libpng_read_fuzzer.cc` harness, how to run it, its implementation rationale, and a comparative analysis of line coverage before and after enhancement.

For reference, we added to our repository the files `utils.h` and `project.diff` to show the modifications that we applied to enhance the existing harness. However, those files are not used when running our first improved harness, as all the modifications are directly pulled from the branch `improved-read-fuzzer` of our forked libpng repository using the following command:

```
git clone -b improved-read-fuzzer --depth 1 \
    https://github.com/gabrielestentella/libpng.git
```

To run the improved harness, move to `part3/improve1` and run the script `./run.improve1.sh` with root privilege. This will create an `oss-fuzz` directory containing the forked libpng repository presented above, build and run the fuzzer for 4 hours and generate the coverage report in the same way as it is done in Part 1.

Enhancements and Rationale: As mentioned in Part 2, the original harness had limited coverage in areas such as colormap processing and metadata accessors. To address that, we first implemented a simple dedicated function, `test_getters`, that was introduced to systematically call the accessor functions after image headers are parsed. Another major improvement was the implementation of `test_simplified_read`, which exercises the simplified read API by changing the image format and enforcing colormap usage. This directly targets uncovered paths like `png_image_read_colormap`. To test the handling of unknown PNG chunks, we defined the function `png_set_read_user_chunk_fn`, which introduces randomized behavior for unknown chunks, in order to force execution through various paths in `pngread.c` and `pngutil.c`. We also apply random transformation through the `apply_random_transforms` function. These include grayscale-to-RGB conversion, palette expansion, alpha stripping, and others, with parameters derived from the input buffer. This stimulates diverse paths within `pngtran.c` and `pngtrans.c`. Additionally, the harness now randomly selects between three pixel data reading methods: `png_read_row`, `png_read_rows`, and `png_read_image`. This variation exercises different internal logic in `pngread.c`. We also introduced `test_png_read_png`, which invokes the higher-level `png_read_png` function and its associated teardown logic (including `png_free_data`), helping to fuzz memory deallocation code paths. Finally, we randomized gamma correction settings across three modes (sRGB, linear, and custom), via `png_set_gAMA`, which adds more variance in how pixel data is interpreted.

Impact on Code Coverage and Further Directions: After 4 hours, this improved version of the read fuzzer showed a significant improvement in overall coverage as shown in Table 1. Most notably, `pngget.c` coverage increased from 3.50% to 81.67% at the line level (from 2.78% to 93.06% at the function level), which validates the effectiveness of `test_getters`. Similarly, line coverage of `pngread.c` increased from 28.26% to 49.33%, thanks to the new simplified API tests, pixel read variations, and unknown

Table 1: Comparison of coverage between the original `libpng_read_fuzzer` and our improved version. The Totals have been readjusted to ignore irrelevant files such as the harness itself.

File	Original read fuzzer		Ours	
	Line Coverage	Function Coverage	Line Coverage	Function Coverage
<code>png.c</code>	37.92% (645/1701)	45.45% (30/66)	56.44% (960/1701)	66.07% (44/66)
<code>pngerror.c</code>	50.00% (190/380)	66.67% (18/27)	59.47% (226/380)	74.07% (20/27)
<code>pngget.c</code>	3.50% (26/742)	2.78% (2/72)	81.67% (606/742)	93.06% (67/72)
<code>pngmem.c</code>	79.28% (88/111)	84.62% (11/13)	79.28% (88/111)	84.62% (11/13)
<code>pngread.c</code>	28.36% (638/2250)	47.37% (18/38)	49.33% (1110/2250)	68.42% (26/38)
<code>pngrio.c</code>	78.57% (11/14)	100.00% (2/2)	78.57% (11/14)	100.00% (2/2)
<code>pngtran.c</code>	30.17% (1025/3397)	52.08% (25/48)	37.83% (1285/3397)	58.33% (28/48)
<code>pngutil.c</code>	72.78% (1898/2608)	93.22% (55/59)	77.76% (2028/2608)	94.92% (56/59)
<code>pngset.c</code>	53.03% (587/1107)	48.98% (24/49)	57.90% (641/1107)	57.14% (28/49)
<code>pngtrans.c</code>	8.67% (36/415)	19.05% (4/21)	34.70% (144/415)	38.10% (8/21)
TOTALS	40.42% (5144/12725)	47.85% (189/395)	55.79% (7099/12725)	73.42% (290/395)

chunk handler. In `png.c`, which encapsulates general-purpose libpng routines, line coverage improved from 37.86% to 56.44%, which is likely due to broader API usage and error handling paths triggered by the input variety introduced by our improved fuzzer. The application of random transformations led to less gains than we expected in `pngtran.c`, but `pngtrans.c` benefited from deeper function testing, including the coverage of private functions like `png_do_invert` and `png_do_strip_channel`. Finally, the function `png_cache_unknown_chunk` in `pngutil.c` was newly covered thanks to the custom chunk handler.

Further improvements could involve generating more sophisticated ancillary chunks (e.g., rich sPLT or iCCP profiles), but this would require a deeper understanding of the PNG format. Other improvements could be to incorporate dictionaries for text chunk fuzzing to construct more complex text chunks (long, short, special characters, different compression flags), or implementing stateful fuzzing logic to explore complex inter-chunk interactions, as some vulnerabilities might only arise from specific sequences or combinations of chunks and API calls.

3.2 Uncovered Region 2: Write Pipeline

To expose the security-critical write pipeline described above, we introduce a new harness, `libpng_write_fuzzer.cc`, placed alongside the standard read-side harness and compiled with the `PNG_WRITE_SUPPORTED` macro enabled. This unlocks access to the full write API, which was previously excluded from the build and thus entirely uninstrumented. We integrate this new harness in OSS-Fuzz’s existing infrastructure so that it can be ran using the same workflow as in Part 1:

```
git clone -b write-fuzzer --depth 1 \
    https://github.com/gabrielestentella/libpng.git
git clone --depth 1 https://github.com/google/oss-fuzz.git

cd oss-fuzz && git apply ../oss-fuzz.diff
```



```

mv ../libpng projects/libpng/
python3 infra/helper.py build_image --no-pull libpng
python3 infra/helper.py build_fuzzers --clean libpng \
    projects/libpng/libpng/
mkdir libpng_write_fuzzer_corpus
python3 infra/helper.py run_fuzzer --corpus-dir \
    ./libpng_write_fuzzer_corpus libpng \
    libpng_write_fuzzer -- -max_total_time=1440

```

Note that the new harness and the modifications to the harness build script are directly clone from our libpng fork. To run this new write fuzzer, move to `part3/improve2` and run the script `./run.improve2.sh` with root privilege.

Enhancements and Rationale: At runtime, the harness creates a `png_struct/``png_info` pair and sets up a custom in-memory write callback to ensure that every short-write error paths remain reachable. It then interprets several bytes from the fuzz buffer to dynamically configure key PNG metadata such as width, height, color type, compression level, filter method, and interlace flag. Optional but security-relevant ancillary chunks (`png_set_sRGB`, `png_set_gAMA_fixed`) are conditionally added, ensuring coverage beyond the PNG core. After calling `png_write_info`, the harness queries the computed rowbyte size, allocates a matching pixel buffer (slab or 2D buffer), and fills it with additional fuzzer input. It concludes the fuzzing path with either a per-row write loop using `png_write_row`, or a single bulk write via `png_write_image`, always finalizing with `png_write_end`. This design ensures that memory allocation and access logic in write-mode code paths are now systematically exercised.

Impact on Code Coverage and Further Directions: After three 4-hour fuzzing campaigns, the coverage metrics show a substantial lift in previously untouched regions: `pngwrite.c` reached 37.24% line and 40.48% function coverage, `pngwutil.c` hit 40.67% / 44.44%, and `pngwio.c` rose to 56.67% / 66.67%. These represent large gains from an initial baseline of 0% across all three files. Although big read-side files still dominates the proportion of covered lines, the specific security gap relative to code that actually serializes attacker-controlled pixel buffers has been reduced from "unreachable" to "over one-third covered", meeting the objectives laid out in Part 2.

In terms of future improvements, several low-cost extensions could further expand the harness's reach. For example, emitting color-type-3 images would trigger `png_set_PLTE` and `png_set_tRNS`, unlocking access to palette and transparency chunk writers. Similarly, populating the `unknown_chunks` table would reach `write_unknown_chunks`, which processes user-defined extensions and often involves complex memory operations. Notice that many of these functions belong to `pngset.c`, a module tightly linked to `pngwrite.c`, as `pngget.c` is to `pngread.c`. These extensions, as simple as they may seem, could prove tricky to integrate in practice, and thus were left untouched. Nonetheless, the current write-side harness already delivers a meaningful security benefit by converting what was a blind spot in libpng's fuzzing profile into a partially explored region. It provides a firm foundation for future refinements aimed at maximizing coverage over this part of libpng.

4 Part 4

Running the improved version of the fuzzer `libpng_read_fuzzer.cc`, we have found a crash provoked by a segmentation fault in two distinct getter functions. To verify the behavior, we have developed a proof of concept (PoC) that reliably triggers the crash.

The code of the PoC (`part4/crash.cc`) takes a PNG file as input (we use the one generated by the fuzzer) and uses it to call the affected libpng functions. To run the PoC:

```
git clone https://github.com/gabrielestentella/CS412-Fuzzing-Lab-libpng.git
cd CS412-Fuzzing-Lab-libpng/part4
./run.poc.sh
```

4.1 Root Cause Analysis

The affected functions are `png_get_sCAL_s` and `png_get_unknown_chunks`, both defined in the file `pngget.c`. The segmentation fault observed in the `png_get_sCAL_s` function arises from a mismanagement of pointers, particularly in the assignment involving the `png_charpp` (alias for `char **`) type. The function signature specifies that the `width` and `height` parameters are of type `png_charpp`. This implies that the caller of the function is expected to pass the addresses of `char *` variables (i.e., `char **pointers`) that the function can use to assign the addresses of internal strings (`scal_s_width` and `scal_s_height`) from the `info_ptr` structure. Inside the function, when the condition guarding the access is satisfied (`png_ptr` and `info_ptr` are non-null, and the `PNG_INFO_sCAL` flag is set in `info_ptr->valid`), the function attempts to copy the internal strings into the caller-provided pointers. This is done by:

```
*width = info_ptr->scal_s_width;
*height = info_ptr->scal_s_height;
```

However, if the caller does not properly initialize the `width` and `height` parameters the dereference operation becomes invalid, leading to a segmentation fault due to writing into an indeterminate or invalid memory location. Likewise, in `png_get_unknown_chunks`, the third parameter, is of type `png_unknown_chunkpp`, which is a typedef for `png_unknown_chunk **`. This implies that the caller is expected to pass the address of a `png_unknown_chunk *`, such that the function can write into it the value of `info_ptr->unknown_chunks`. Within the function body, the assignment:

```
*unknowns = info_ptr->unknown_chunks;
```

directly dereferences the `unknowns` pointer to write the internal chunk data. This operation is safe only if the pointer `unknowns` is itself pointing to a valid memory location where a `png_unknown_chunk *` can be stored. If this precondition is not met then the dereference operation results in a segmentation fault.

4.2 Security Implication

The security implications of these bugs, which involve unsafe dereferencing of caller-supplied double pointers, are primarily related to stability rather than direct exploitation. These issues can lead to segmentation faults if the API is misused, such as by passing uninitialized or pointers, but the bugs are not exploitable on their own. The memory writes occur only if the caller fails to follow the API contract, and the attacker does not have direct control over the target memory address or the data being written. As such, while a crash (i.e., denial of service) can be reliably triggered in applications that incorrectly use these APIs, there is no immediate primitive for arbitrary code execution or memory corruption that an attacker can leverage without additional, unrelated vulnerabilities. The direct control over the dereferenced pointer value is limited, so it is highly unlikely that these bugs lead to arbitrary pointer overwrites, enabling information

disclosure, arbitrary code execution, or control flow hijacking. We consider these bugs non-exploitable in isolation under typical threat models.

We provide a fix of the affected functions at the fork of the `libpng` repository that we also used to test out improved fuzzer, i.e. the branch `improved-read-fuzzer` of <https://github.com/gabrielestentella/libpng.git>.